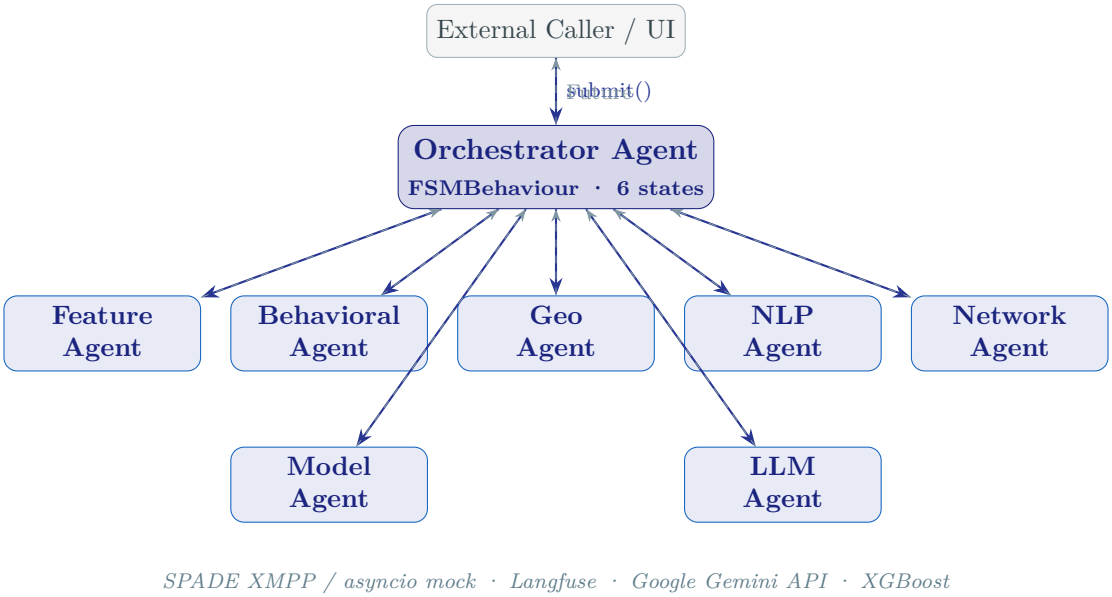
Laboratory assignment

Component 3

Authors: Harangus Paul Andrei and Lupu Stefan
Group: ICA2

April 22, 2026



Framework:	SPADE (Smart Python Agent Development Environment)
Observability:	Langfuse — full LLM call tracing and token metrics
ML Backend:	XGBoost ensemble + Google Gemini (two-tier LLM)
Agents:	8 total (1 orchestrator · 5 specialist analysis · 1 model · 1 LLM)
Comm. model:	Async message passing, JID addressing, thread-correlated batches
Dual backend:	Real XMPP (distributed) or asyncio mock (in-process, Windows)

Contents

1 System Overview and Architecture **3**
1.1 Agent Roster . . . 3
1.2 High-Level Data Flow . . . 3
1.3 Execution Modes . . . 3

2 Conceptual Agent Modelling — PAGES Framework **4**
2.1 Agent 1 — OrchestratorAgent . . . 5
2.1.1 FSM State Descriptions and Transition Logic . . . 6
2.1.2 SPADE Integration Details . . . 6
2.2 Agent 2 — FeatureAgent . . . 8
2.2.1 Feature Engineering — Detailed Column Specification . . . 8
2.2.2 SPADE Integration Details . . . 9
2.3 Agent 3 — BehavioralAgent . . . 10
2.3.1 Behavioral Signal Computation . . . 10
2.3.2 Communication and SPADE Integration . . . 10
2.4 Agent 4 — GeoAgent . . . 11
2.4.1 Geo Signal Computation . . . 11
2.4.2 SPADE Integration Details . . . 11
2.5 Agent 5 — NLPAgent . . . 12
2.5.1 NLP Signal Computation . . . 12
2.5.2 SPADE Integration Details . . . 12
2.6 Agent 6 — NetworkAgent . . . 14
2.6.1 Network Signal Computation . . . 14
2.6.2 SPADE Integration Details . . . 14
2.7 Agent 7 — ModelAgent . . . 15
2.7.1 Scoring Logic and Fallback . . . 15
2.7.2 XGBoost Hyperparameters . . . 15
2.8 Agent 8 — LLMAgent . . . 17
2.8.1 Two-Tier Gemini Escalation . . . 17
2.8.2 Evidence Bundle (21 fields) — llm_orchestrator.py:93 . . . 17
2.8.3 Langfuse Tracing . . . 17
2.8.4 Fraud Memory Feedback Loop . . . 18

3 Properties of the Environment **19**
3.1 Theoretical Classification . . . 19
3.2 Environmental Components . . . 19
3.3 Configuration Parameters (`fraud_mas/config.py`) . . . 20
3.4 Environment Constraints and Failure Modes . . . 20

4 Application Design — Class Diagram **22**
4.1 Core Inheritance Hierarchy . . . 22
4.2 SPADE Infrastructure Classes . . . 22
4.3 Analysis Module Functions (called by agents) . . . 23
4.4 Pipeline Wrapper Classes . . . 23

5 Communication and Interaction Model **24**
5.1 Message Protocol Specification . . . 24
5.2 Full Message Type Catalogue . . . 24
5.3 Interaction Sequence Diagram . . . 25
5.4 Timeout and Fault-Tolerance Protocol . . . 25
5.5 Message Correlation and Routing . . . 25
5.6 SPADE Dual-Backend Comparison . . . 26
5.7 Complete Data Flow Summary . . . 26

6 SPADE Framework Integration **27**
6.1 Dual Backend Architecture . . . 27
6.2 Mock Backend: Global Mailbox Bus . . . 27
6.3 Agent Lifecycle . . . 27
6.4 XMPP Server Configuration . . . 27
6.5 Agent JIDs and XMPP Addresses . . . 28

7 Langfuse Observability Integration **29**
7.1 Purpose and Architecture . . . 29
7.2 Initialisation and Credentials . . . 29
7.3 Trace and Generation Structure . . . 29
7.4 Safety Wrappers . . . 29
7.5 What Langfuse Captures . . . 30
7.6 Fraud Memory Feedback Loop and Langfuse Integration . . . 30

SYSTEM OVERVIEW AND ARCHITECTURE

The Fraud Detection MAS is a distributed, event-driven pipeline for classifying financial payment transactions as **fraud** or **legit**. It consists of eight autonomous SPADE agents collaborating through structured asynchronous message passing. The architecture is hierarchical: a single **OrchestratorAgent** implements a Finite State Machine that governs the entire pipeline lifecycle, while six specialist agents operate as independent **CyclicBehaviour** responders. A two-tier large language model (Google Gemini) resolves borderline cases, with all LLM interactions traced via **Langfuse**.

Agent Roster

Agent	Behaviour	JID	primary Responsibility
OrchestratorAgent	FSMBehaviour	`orchestrator@localhost`	Pipeline coordination, fan-out, merge, routing
FeatureAgent	CyclicBehaviour	`feature_agent@localhost`	Statistical feature engineering (20+ columns)
BehavioralAgent	CyclicBehaviour	`behavioral_agent@localhost`	Velocity and behavioural anomaly scoring
GeoAgent	CyclicBehaviour	`geo_agent@localhost`	Geolocation risk, impossible travel detection
NLPAgent	CyclicBehaviour	`nlp_agent@localhost`	Text phishing, keyword and obfuscation risk
NetworkAgent	CyclicBehaviour	`network_agent@localhost`	Graph-based fraud ring and shared-infra detection
ModelAgent	CyclicBehaviour	`model_agent@localhost`	XGBoost scoring, threshold classification
LLMAgent	CyclicBehaviour	`llm_agent@localhost`	Gemini LLM final decision + memory update

High-Level Data Flow

Raw transaction CSVs and associated JSON records (users, locations, SMS, emails) are merged by `load_and_merge_dataset()` into a single enriched DataFrame. This is serialised to `list[dict]` and submitted to the Orchestrator. The Orchestrator sequentially routes the batch through Feature Engineering, then fans out in parallel to Behavioral, Geo, NLP, and Network agents, merges their signal columns, routes to the ModelAgent for scoring, and optionally routes borderline rows to the LLMAgent. Results are returned to the caller as a resolved `asyncio.Future`.

Execution Modes

The system supports two modes with identical inputs and outputs:

- **Direct Pipeline** (`pipeline.py`): Agent logic replaced by direct module calls inside a `ThreadPoolExecutor`. Used by the *Detect* page in the Streamlit UI. No SPADE overhead, no XMPP dependency.
- **SPADE Agent Pipeline** (`spade_pipeline.py`): Full MAS with message-passing agents. Used by the *SPADE Agents* page. Supports real XMPP (distributed) or asyncio mock (in-process). Wrapped in a thread via `run_spade_pipeline_sync()` for Streamlit compatibility.

CONCEPTUAL AGENT MODELLING — PAGES FRAMEWORK

Each agent is characterised along the PAGES dimensions: **P**ercepts · **A**ctions · **G**oals · **E**nvironment interaction · **S**elf-model (internal behaviour structure and state).

Agent 1 — OrchestratorAgent

PAGES — OrchestratorAgent

Percepts

(1) A batch of raw transaction dictionaries dequeued from the internal `asyncio.Queue _inbox`; each element is a `dict` with at minimum `transaction_id`, `amount`, `sender_id`, `recipient_id`, and `timestamp`. (2) Reply messages from all specialist agents carrying `metadata["thread"] == batch_id` (batch UUID), `metadata["type"]` distinguishing the reply kind (`feature_result`, `signals`, `scores`, `decisions`), and `message.body` containing JSON-encoded payloads. (3) A timeout signal (implicit: `asyncio.TimeoutError` on `receive()`) that triggers graceful degradation.

Actions

(1) **Fan-out**: construct and `send()` a `Message(type="analyze", thread=bid)` to each target agent JID via the SPADE/mock mailbox bus. (2) **Signal merge**: left-join incoming `signals` DataFrames on `transaction_id` into the running batch DataFrame. (3) **Score routing**: split the scored batch into `fraud/legit` (auto-decided) vs. `review` (sent to LLMAgent). (4) **Finalization**: iterate over all rows, look up `model_score`, `initial_label`, and `llm_decisions`; populate `label` and `llm_reason`; call `ctx["future"].set_result(results)` to unblock the caller. (5) **State transition**: call `set_next_state()` based on context (skip `LLM_REVIEW` if no review cases).

Goals

(1) Guarantee every transaction in the batch receives a definitive `label` (`"fraud"` or `"legit"`) and is included in the resolved Future result, with no row silently dropped. (2) Enforce the batch deadline: use `AGENT_TIMEOUT=30 s` per specialist agent and `AGENT_TIMEOUT*2=60 s` for the LLM; on timeout substitute default zero-signal rows and continue. (3) Maintain strict FSM semantics: transitions follow the registered graph; no out-of-graph transition is taken. (4) Preserve correlation: all outgoing messages carry the same `thread=batch_id`; all incoming `receive()` calls use a `Template` matched on this thread.

Environment

Reads from the internal `asyncio.Queue _inbox` (written by external callers via `submit()`). Writes messages to the SPADE/asyncio mailbox bus (`_MAILBOXES[jid]`). Maintains a mutable per-batch context dictionary `self._current` that persists across FSM state transitions within a single pipeline run: `{batch_id, rows, future, signals, scores, review_rows, review_ids, llm_decisions}`.

Self-model

FSMBehaviour (real SPADE or mock) with six states wired via `add_state()` and `add_transition()`. Initial state: `IDLE`. All six states share the FSM's single `_queue` so `receive()` calls inside any state read the same mailbox. The agent registers the FSM during `setup()` by calling `self.add_behaviour(fsm)`. SPADE's dispatch loop (`_dispatch_loop`) routes incoming raw messages to the FSM queue, which each state reads via `await self.receive(timeout)`.

FSM State Descriptions and Transition Logic

State	Behaviour	Sends	Awaits / Next State
IDLE	`asyncio.wait_for(_inbox.get(), 1.0s)`; on timeout loops back to IDLE; on success initialises `self._current` with `batch_id`, rows, future, empty signals dict	Nothing	→ `FEATURE` on batch available
FEATURE	Builds and sends `analyze` request to `feature_agent@localhost`; waits for reply; on timeout logs warning and passes raw rows through	`type=analyze` to FeatureAgent	→ `PARALLEL` always
PARALLEL	Iterates `_PARALLEL_AGENTS` and calls `send()` for each; then collects up to 4 `signals` replies within a single deadline window; merges each via `DataFrame.merge(on="transaction_id")`	`type=analyze` to Behavioral, Geo, NLP, Network (4 messages)	→ `SCORE` after deadline
SCORE	Sends fully enriched rows to `model_agent`; on reply splits by `initial_label`; populates `ctx["review_rows"]` and `ctx["review_ids"]`	`type=score` to ModelAgent	→ `LLM_REVIEW` if review rows; → `FINALIZE` otherwise
LLM_REVIEW	Sends review subset to `llm_agent` with `timeout=AGENT_TIMEOUT*2`; on timeout synthesises default `{label:"fraud", llm_reason:"timeout"}` for every review row	`type=llm_decide` to LLMAgent	→ `FINALIZE` always
FINALIZE	Builds `results` by iterating all rows, looking up `score_map` and `llm_map`, merging into final dicts; calls `ctx["future"].set_result(results)`; resets `self._current`	Nothing	→ `IDLE`

SPADE Integration Details

The Orchestrator is the only agent that uses **FSMBehaviour**. All six state classes inherit from SPADE's `State` (or the mock equivalent). Each state accesses the parent agent object via `self.agent: OrchestratorAgent`. SPADE's `_dispatch_loop` routes incoming messages to the FSM's shared `_queue`; state-level `receive(timeout)` reads from that queue. For XMPP transport, `FraudBaseAgent._async_connect()` disables TLS and enables PLAIN auth for the local Prosody server. The `submit()` method is safe to call from any thread: it calls `self._inbox.put_nowait()` (thread-safe asyncio queue operation).

Orchestrator: setup() — FSM construction (orchestrator.py:271)

```
async def setup(self):
    fsm = FSMBehaviour()
    fsm.add_state(name=S_IDLE,       state=IdleState(),      initial=True)
    fsm.add_state(name=S_FEATURE,    state=FeatureState())
    fsm.add_state(name=S_PARALLEL,   state=ParallelState())
    fsm.add_state(name=S_SCORE,      state=ScoreState())
    fsm.add_state(name=S_LLM_REVIEW, state=LLMReviewState())
    fsm.add_state(name=S_FINALIZE,   state=FinalizeState())
    fsm.add_transition(S_IDLE, S_IDLE);   fsm.add_transition(S_IDLE, S_FEATURE)
    fsm.add_transition(S_FEATURE, S_PARALLEL)
    fsm.add_transition(S_PARALLEL, S_SCORE)
    fsm.add_transition(S_SCORE, S_LLM_REVIEW); fsm.add_transition(S_SCORE, S_FINALIZE)
    fsm.add_transition(S_LLM_REVIEW, S_FINALIZE)
    fsm.add_transition(S_FINALIZE, S_IDLE)
    self.add_behaviour(fsm)
```

submit() — external API (orchestrator.py:260)

```
def submit(self, rows: list[dict]) -> asyncio.Future:
    loop    = asyncio.get_event_loop()
    future  = loop.create_future()
```

```
    batch_id = str(uuid.uuid4())[:8]
    self._inbox.put_nowait((batch_id, rows, future))
    return future
```

Agent 2 — FeatureAgent

PAGES — FeatureAgent

Percepts	(1) A `Message` with `metadata["performative"]="request"`, `metadata["type"]="analyze"`, and `body` containing a JSON-encoded `list[dict]` of raw transaction records. (2) `LabelEncoder` objects pre-loaded from `artifacts/label_encoders.pkl` at agent `setup()` into `self.encoders: dict`. If the file does not exist, `encoders` is an empty dict and all categorical columns default to 0. (3) Optionally: transaction columns `balance_after`, `sender_id`, `recipient_id`, `timestamp` — the function gracefully handles their absence with default zero values.
Actions	(1) Decode JSON body to a `pd.DataFrame`. (2) Call `engineer_features(df, encoders=self.encoders, fit=False)` from `fraud_mas/features.py` — this mutates and returns a copy with all derived columns appended. (3) Serialise the result via `df_out.to_dict(orient="records")`. (4) Construct a reply message via `make_reply(msg, payload, "feature_result")` setting `performative="inform"`. (5) On any exception: log the error and pass through the original raw rows unchanged.
Goals	Produce a normalised, encoded feature matrix for all downstream agents and the XGBoost model. Ensure backward compatibility: if an encoder for a column is absent, default-encode to zero rather than raising an error. Introduce no information leakage: `fit=False` prevents refitting on inference data.
Environment	*Stateful* — `self.encoders` persists across requests, loaded once at `setup()` and reused for every batch. Uses pandas `groupby`, `sort_values`, and `merge` operations over the transaction history within each received batch for per-sender statistics.
Self-model	**CyclicBehaviour** (inner class `AnalyseBehaviour`) registered in `setup()` with a `Template(performative="request", type="analyze")`. The behaviour calls `await self.receive(timeout=5)` in each iteration; if no message arrives it returns immediately and the loop continues.

Feature Engineering — Detailed Column Specification

Amount Features — features.py:40–66

Column	Formula / Description
`amount_log`	$\ln(1 + \texttt{amount} + 1.0)$ — log-scale amount
`amount_zscore`	$(\texttt{amount} - \mu_s)/(\sigma_s + 10^{-6})$ where μ_s, σ_s are per-sender mean/std
`balance_ratio`	$\texttt{amount}/(\lvert\texttt{balance_after}\rvert + 10^{-6})$ — fraction of balance spent
`amount_vs_user_mean`	$\texttt{amount}/(\mu_s + 10^{-9})$ — ratio to personal average
`is_personal_record`	1 if $\texttt{amount} = \max_s(\texttt{amount})$ (personal max for this sender)
`user_tx_count`	Total transactions by this sender in the batch
`user_mean_amount`	μ_s — sender's mean transaction amount
`user_std_amount`	σ_s — sender's standard deviation of amounts

Temporal Features — features.py:72–81

Column	Formula / Description
`hour`	`timestamp.dt.hour` $\in [0, 23]$
`hour_sin`	$\sin(2\pi \cdot \texttt{hour}/24)$ — cyclic encoding
`hour_cos`	$\cos(2\pi \cdot \texttt{hour}/24)$ — cyclic encoding
`day_of_week`	`timestamp.dt.dayofweek` $\in [0, 6]$ (Monday=0)
`month`	`timestamp.dt.month` $\in [1, 12]$
`is_weekend`	1 if `day_of_week` ≥ 5
`is_night`	1 if `hour` < 6 or `hour` ≥ 22

Velocity, Recipient, and Card-Test Features — features.py:87–130

Column	Formula / Description
`tx_count_1h`	Rolling count of transactions by the same sender in the past 1 hour
`tx_count_24h`	Rolling count of transactions by the same sender in the past 24 hours
`new_recipient`	1 if the (sender, recipient) pair has not appeared in any earlier row (set tracking)
`card_test_score`	1.0 if prev_amount < 10, ratio > 50, and time < 60 min; 0.7 if ratio > 20; 0.3 if ratio > 10; else 0
`tx_type_enc`	`LabelEncoder.transform(transaction_type)`
`payment_method_enc`	`LabelEncoder.transform(payment_method)`
`merchant_enc`	`LabelEncoder.transform(merchant)` (top-50 capped)
`country_enc`	`LabelEncoder.transform(country)` (top-30 capped)
`device_type_enc`	`LabelEncoder.transform(device_type)`

SPADE Integration Details

The inner class `AnalyseBehaviour(CyclicBehaviour)` is defined within `FeatureAgent` and registers itself in `setup()` with a `Template` that matches on both `performative="request"` and `type="analyze"`. The template filter ensures the cyclic behaviour ignores all other message types routed through the same agent mailbox. The `LabelEncoders` are loaded once via `load_label_encoders()` (which deserialises `artifacts/label_encoders.pkl`) and stored in `self.encoders`. This means the Feature Agent is the only truly stateful specialist agent. `make_reply(msg, payload, "feature_result")` calls `msg.make_reply()` which automatically copies the `thread` field, preserving batch correlation.

Agent 3 — BehavioralAgent

PAGES — BehavioralAgent

Percepts	A `Message` with `type="analyze"` whose body contains the enriched transaction list produced by FeatureAgent. Relevant input columns consumed: `sender_id`, `recipient_id`, `amount`, `timestamp`, `user_mean_amount`. All columns are optional — missing ones produce safe zero-filled defaults.
Actions	(1) Decode the JSON body to `pd.DataFrame`. (2) Call `compute_behavioral_signals(df)` from `fraud_mas/behavioral.py`. (3) Compute rolling velocity counts using `groupby` and sorted timestamps. (4) Detect novel (sender, recipient) pairs using in-memory set tracking. (5) Flag large jumps: `amount` $> 3 \times$ `user_mean_amount`. (6) Compute composite `behav_score`. (7) Send `type="signals"` reply with `metadata["agent"]="behavioral"` so the Orchestrator can identify the sender in its parallel collection loop.
Goals	Detect deviations from each user's historical transaction behaviour within the current batch. Provide a normalised composite risk score $\in [0, 1]$ that the ModelAgent and LLMAgent can use directly. Flag burst activity and novel routing which are canonical indicators of account compromise and money mule behaviour.
Environment	*Stateless* per invocation. Relies exclusively on the rows contained in the current batch for rolling window computations — no persistent user history is maintained between batches (this is a known limitation; within-batch velocity is a proxy for real-time history).
Self-model	**CyclicBehaviour** with `Template(performative="request", type="analyze")`. Receives a message, calls the analysis function, sends the reply, and loops. The `agent_name = "behavioral"` field is injected into `metadata["agent"]` by `make_reply()` to allow the Orchestrator's `ParallelState` to identify the reply source.

Behavioral Signal Computation

Signal Columns and Composite Score Formula

Column	Computation	Range	Weight
`behav_velocity_1h`	Count of rows where same `sender_id` and $\lvert\Delta t\rvert \leq 1\,\text{h}$	$\mathbb{Z}^+$	0.30
`behav_velocity_24h`	Count of rows where same `sender_id` and $\lvert\Delta t\rvert \leq 24\,\text{h}$	$\mathbb{Z}^+$	0.15
`behav_new_recipient`	1 if (sender, recipient) pair first seen in sorted batch	$\{0, 1\}$	0.30
`behav_large_jump`	1 if `amount` $> 3 \times$ `user_mean_amount`	$\{0, 1\}$	0.25
`burst_score`	$\min(\texttt{velocity_1h}/5,\ 1.0)$	$[0, 1]$	—

$$\textbf{behav_score} = \min\Big(0.30 \cdot v_{1h} + 0.15 \cdot v_{24h} + 0.30 \cdot r_{new} + 0.25 \cdot j_{large},\ 1.0\Big)$$

Communication and SPADE Integration

The BehavioralAgent uses a **CyclicBehaviour** inner class. In `setup()`, it registers this behaviour with a `Template` that matches on `type="analyze"` and `performative="request"`. When the Orchestrator's `ParallelState` sends four simultaneous `analyze` messages, the BehavioralAgent's dispatch loop routes the message to the behaviour's `_queue`. The reply is constructed with `make_reply(msg, payload, "signals")` which auto-copies `msg.thread` (the batch UUID) and sets `metadata["agent"]="behavioral"`. In `ParallelState`, the Orchestrator filters incoming replies by checking `reply.metadata.get("type") == "signals"` and `reply.metadata.get("agent") in _PARALLEL_AGENTS`. Only the columns not already in the base DataFrame are merged via left-join on `transaction_id`.

Agent 4 — GeoAgent

PAGES — GeoAgent

Percepts	A `Message(type="analyze")` whose body contains transactions enriched with geographic fields merged from `locations.json`: `latitude`, `longitude`, `country` (ISO2 code), and `timestamp`. Also reads `sender_id` for per-sender home location computation. All geo fields default to 0 if absent.
Actions	(1) Parse DataFrame from JSON body. (2) Call `compute_geo_signals(df)` from `fraud_mas/geo.py`. (3) Compute the median home latitude/longitude per sender using `groupby().median()`. (4) Apply the Haversine formula to each row using its coordinates vs. the sender's home point. (5) Sort rows by sender and timestamp; compute time delta between consecutive transactions for the same sender and derive km/h speed; flag values > 900 km/h as impossible travel. (6) Flag the ISO2 country code against the hard-coded high-risk set. (7) Compute composite `geo_score` $\in [0, 1]$ and send `type="signals"` reply.
Goals	Identify geographically implausible transactions (impossible speed of movement), transactions from high-risk jurisdictions, and transactions that deviate significantly from a user's inferred home region. Provide a normalised $[0, 1]$ risk signal independent of other agents.
Environment	*Stateless* per invocation. The sender's home location is computed as the median of coordinates within the current batch — no cross-batch history is maintained. The high-risk country set is a hard-coded module-level constant.
Self-model	**CyclicBehaviour** with `Template(performative="request", type="analyze")`. Agent name `"geo"` is embedded in reply metadata so the Orchestrator's `ParallelState` can identify the signal source.

Geo Signal Computation

Signal Columns and Haversine Formula

Column	Computation	Range	Weight
`geo_high_risk_country`	1 if `country` $\in$ {NG, RU, CN, KP, IR, VE, BY, UA, RO, PK}	$\{0, 1\}$	0.35
`geo_distance_km`	Haversine$((\texttt{lat}, \texttt{lon}),\ \tilde{\mu}_s)$ where $\tilde{\mu}_s$ = sender median lat/lon	$\mathbb{R}^+$	0.30
`geo_impossible_travel`	1 if `geo_distance_km`$/\Delta t_h > 900$ between consecutive rows of same sender	$\{0, 1\}$	0.35

$$\textbf{geo_score} = \min\Big(0.35 \cdot c_{risk} + 0.30 \cdot \min(d_{km}/5000, 1) + 0.35 \cdot t_{imp},\ 1.0\Big)$$

The Haversine formula used:

$$d = 2R \arcsin\left(\sqrt{\sin^2\frac{\Delta\phi}{2} + \cos\phi_1 \cos\phi_2 \sin^2\frac{\Delta\lambda}{2}}\right), \qquad R = 6371\,\text{km}$$

where ϕ = latitude in radians, λ = longitude in radians.

SPADE Integration Details

Identical lifecycle to FeatureAgent. The behaviour registers with a `Template` filtered on `type="analyze"`. In the asyncio mock backend, the GeoAgent's JID `geo_agent@localhost` maps to a dedicated `asyncio.Queue` in the global `_MAILBOXES` dict. The Orchestrator's `ParallelState` sends a message to this JID and the mock's `_dispatch_loop` routes it to the behaviour's `_queue`. The behaviour emits a `signals` reply with `metadata["agent"]="geo"`, which the Orchestrator uses to filter and merge the geo-specific columns.

Agent 5 — NLPAgent

PAGES — NLPAgent

Percepts	A `Message(type="analyze")` body containing transactions augmented with text fields merged from `sms.json` (`sms_text`: all SMS messages for the transaction or user concatenated) and `mails.json` (`mail_text`: email subject + body). Also reads `description` and `merchant` from the base transaction record. All text fields default to empty strings.
Actions	(1) Call `compute_nlp_signals(df)` from `fraud_mas/nlp_risk.py`. (2) Tokenise each text field to lowercase; compute keyword hit fraction over a 25-term risk vocabulary. (3) Apply compiled regex patterns against the 9 phishing URL templates (case-insensitive). (4) Compute the fraction of non-alphanumeric characters as an obfuscation metric. (5) Detect all-caps strings of length > 3. (6) Compute per-source (`description`, SMS, email) subscores and aggregate into `nlp_score`. (7) Send `type="signals"` reply with `metadata["agent"]="nlp"`.
Goals	Detect social-engineering communication patterns that co-occur with fraudulent transactions: phishing URLs in linked messages, urgency/cryptocurrency/gift-card language, and text obfuscation often used to evade simple keyword filters. Operate robustly on missing text fields (empty string defaults).
Environment	*Stateless* per invocation. Risk vocabulary and phishing regex patterns are module-level constants compiled once at import time. No external API calls or cross-batch state.
Self-model	**CyclicBehaviour** with `Template(performative="request", type="analyze")`. All string operations use vectorised pandas `str` methods to minimise latency within the asyncio event loop.

NLP Signal Computation

Signal Columns, Risk Vocabulary, and Phishing Patterns

Column	Computation	Range	Weight
`nlp_keyword_score`	min (keyword hit count/token count, 1.0) over `description`	$[0, 1]$	0.35
`nlp_phishing`	1 if any phishing regex matches the description/text	$\{0, 1\}$	0.20
`nlp_obfuscation`	$\lvert\{c \in \text{text} : c \notin [\text{a-z0-9}]\}\rvert / \lvert\text{text}\rvert$	$[0, 1]$	0.10
`nlp_all_caps`	1 if `text.isupper()` and `len(text)` > 3	$\{0, 1\}$	0.10
`nlp_sms_score`	$\max(\text{keyword_risk}, \text{phishing})$ computed on `sms_text`	$[0, 1]$	0.15
`nlp_mail_score`	$\max(\text{keyword_risk}, \text{phishing})$ computed on `mail_text`	$[0, 1]$	0.10
`phishing_link_count`	Cumulative count of phishing regex matches across all text fields	$\mathbb{Z}^+$	—

nlp_score $= \min(0.35k + 0.20p + 0.10o + 0.10c + 0.15s + 0.10m,\ 1.0)$

Risk vocabulary (25 terms): casino, gambling, crypto, bitcoin, forex, investment, loan, offshore, anonymous, escrow, wire, transfer, gift card, prepaid, reload, moneygram, western union, urgent, verify, account, suspended, click, confirm, free money, limited offer

Phishing URL regex patterns: `paypa1`, `amaz0n`, `dhl-delivr`, `bitly/`, `verify.*secure`, `secure.*verify`, `free.*money`, `click.*here`, `account.*suspend`

SPADE Integration Details

The NLPAgent follows the same CyclicBehaviour pattern. All string processing uses `pd.Series.str` vectorised operations, so `run()` completes without releasing the asyncio event loop unnecessarily. The reply carries `metadata["agent"]="nlp"` and `metadata["type"]="signals"` so that the Orchestrator's

`ParallelState` can identify and merge the NLP-specific signal columns into the batch DataFrame.

Agent 6 — NetworkAgent

PAGES — NetworkAgent

Percepts	A `Message(type="analyze")` body containing transactions enriched with: `sender_id`, `recipient_id`, `device_id` (from user records), `ip_address` (from user/location records). All fields are optional; missing values produce zero-valued signals.
Actions	(1) Call `compute_network_signals(df)` from `fraud_mas/network_analysis.py`. (2) Build a directed weighted transaction graph $G = (V, E)$ where V = all sender/recipient IDs and E = transactions, weighted by count. (3) Compute out-degree per sender (unique recipients) from the graph. (4) Build shared-attribute graphs (device_id, ip_address) and count peers for each sender. (5) Compute community risk: for each sender, find all recipients and calculate the mean in-degree of those nodes in G, clipped at 10. (6) Flag new nodes: senders with ≤ 2 total transactions. (7) Send `type="signals"` reply with `metadata["agent"]="network"`.
Goals	Uncover structural fraud patterns: money mule rings (high shared-device clusters), coordinated attacks (shared IP), hub-and-spoke fraud rings (high sender out-degree), and new synthetic account activity (new node flag). Provide graph-derived risk signals unavailable from tabular feature engineering alone.
Environment	*Stateless* per invocation. The graph is constructed fresh from the current batch only. This limits detection to within-batch ring structures; cross-batch graph persistence is a future extension. Uses `networkx.DiGraph`.
Self-model	**CyclicBehaviour** with `Template(performative="request", type="analyze")`. Agent name `"network"` embedded in reply metadata for Orchestrator identification.

Network Signal Computation

Signal Columns and Graph Construction

Column	Computation	Range	Weight
`net_shared_device`	$\lvert\{s' \neq s : \texttt{device_id}_{s'} = \texttt{device_id}_s\}\rvert$	$\mathbb{Z}^+$	0.20
`net_shared_ip`	$\lvert\{s' \neq s : \texttt{ip}_{s'} = \texttt{ip}_s\}\rvert$	$\mathbb{Z}^+$	0.20
`net_degree`	$\text{outdeg}_G(s) = \lvert\{r : (s,r) \in E\}\rvert$	$\mathbb{Z}^+$	0.20
`net_community_risk`	$\min\Big(\text{mean}_{r \in N^+(s)}(\text{indeg}_G(r)),\ 10\Big)/10$	$[0,1]$	0.20
`net_new_node_score`	1 if $\sum_r \mathbf{1}[(s,r) \in E] \leq 2$	$\{0,1\}$	0.20

$$\textbf{net_score} = \min\Big(\tfrac{1}{5}(d_{dev} + d_{ip} + d_{net}/10 + r_{comm} + n_{new}),\ 1.0\Big)$$

Graph G is a `networkx.DiGraph`. Directed edges: `sender_id` → `recipient_id` with `weight` = transaction count. Additional undirected shared-attribute graphs are built for device_id and ip_address to compute `net_shared_device` and `net_shared_ip` via node adjacency lookup.

SPADE Integration Details

The NetworkAgent follows the same CyclicBehaviour registration pattern. Because `networkx` graph construction is CPU-bound, it runs synchronously in the asyncio loop — acceptable for batch sizes typical of fraud pipelines ($\leq$ 10,000 rows). For large batches, the operation could be offloaded to a `ThreadPoolExecutor` via `asyncio.get_event_loop().run_in_executor()`. The reply carries `metadata["agent"]="network"` and `metadata["type"]="signals"`.

Agent 7 — ModelAgent

PAGES — ModelAgent

Percepts	A `Message(type="score")` whose body is a JSON-encoded list of fully enriched transaction dicts containing all feature columns from FeatureAgent plus all signal columns (`behav_score`, `geo_score`, `nlp_score`, `net_score`) from the four parallel agents. Also percepts: `self.model` (XGBClassifier or None, loaded once at `setup()`) and the feature column list from `artifacts/feature_cols.pkl`.
Actions	(1) Decode JSON body to `pd.DataFrame`. (2) Call `predict_proba(df, self.model)`: if a trained model is available, load the exact feature column order from `artifacts/feature_cols.pkl` (or from `model.get_booster().feature_names`), align the DataFrame columns, fill missing with 0, and call `model.predict_proba(X)[:, 1]`. If no model, fall back to `rule_based_score(df)` using the weighted ensemble. (3) Call `apply_thresholds(scores)`: assign `"fraud"` for score ≥ 0.75, `"legit"` for score ≤ 0.25, `"review"` otherwise. (4) Build and send a `type="scores"` reply with payload `[transaction_id, model_score, initial_label, ...]`. (5) On exception: emit default `{model_score: 0.5, initial_label: "review"}` for every row.
Goals	Produce a calibrated per-transaction fraud probability $\in [0, 1]$ and a three-way preliminary classification. Guarantee no row is silently dropped on exception. Correctly align feature columns to the exact order seen during training to prevent XGBoost feature mismatch errors.
Environment	*Stateful* — `self.model` (XGBClassifier) is loaded once at `setup()` via `load_model()` which deserialises `models/xgb_model.pkl`. If the file is absent, `self.model = None` and the rule-based fallback is used.
Self-model	**CyclicBehaviour** (inner class `ScoreBehaviour`) with `Template(performative="request", type="score")`. Message type is `"score"` not `"analyze"` — the Orchestrator uses a different type for this request to separate it from the parallel signal-collection round.

Scoring Logic and Fallback

Threshold Classification and Rule-Based Ensemble

Threshold table (`model.py:100`):

Score	initial_label	Disposition
≥ 0.75	`"fraud"`	Auto-flag; no LLM
≤ 0.25	`"legit"`	Auto-clear; no LLM
$]0.25, 0.75[$	`"review"`	$\rightarrow$ LLMAgent

Rule-based fallback weights (`model.py:48`):

Feature	Weight
`behav_score`	0.22
`card_test_score`	0.20
`geo_score`	0.15
`net_score`	0.13
`nlp_score`	0.12
`amount_zscore`	0.08
`new_recipient`	0.06
`is_night`	0.04

XGBoost Hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
`n_estimators`	400	`max_depth`	6	`learning_rate`	0.05
`subsample`	0.8	`colsample_bytree`	0.8	`eval_metric`	logloss
`random_state`	42	`use_label_encoder`	False	`verbose`	False

Full feature column set (`ALL_FEATURES` = `FEATURE_COLS` ∪ `AGENT_SCORE_COLS`): the 22 `FEATURE_COLS` (amount_log, amount_zscore, balance_ratio, amount_vs_user_mean, is_personal_record, hour, hour_sin,

hour_cos, day_of_week, month, is_weekend, is_night, tx_count_1h, tx_count_24h, new_recipient, card_test_score, user_tx_count, user_mean_amount, user_std_amount, tx_type_enc, payment_method_enc) plus the 4 agent scores (behav_score, geo_score, nlp_score, net_score).

Agent 8 — LLMAgent

PAGES — LLMAgent

Percepts

(1) A `Message(type="llm_decide")` whose body contains only transactions with `initial_label="review"`, enriched with `model_score` attached by the Orchestrator's `ScoreState`. (2) `fraud_memory.json` loaded from `artifacts/fraud_memory.json` via `load_fraud_memory()`: a dict with key `"fraud_patterns"`: a list of up to 50 natural-language fraud reason strings, accumulated from prior batches. (3) The Google Gemini API response (JSON), parsed into `{"decision", "confidence", "reason"}`. (4) Langfuse trace/generation objects (optional) for observability.

Actions

(1) Call `llm_decide(rows)` from `fraud_mas/llm_orchestrator.py`. (2) Call `load_fraud_memory()` and `build_system_prompt(memory)` to inject known patterns. (3) For each row: call `build_evidence_bundle(row)` to build a 21-field JSON evidence string. (4) Call `_call_llm(LLM_MODEL_CHEAP, system, evidence)`: `GenerateContentConfig(temperature=0.0, max_output_tokens=256, response_mime_type="application/json")`. (5) If `confidence < LLM_CONFIDENCE_MIN (0.70)`: escalate to `_call_llm(LLM_MODEL_STRONG, ...)`. (6) Update `fraud_memory.json` with new fraud reasons; cap at 50 entries. (7) Flush Langfuse; send `type="decisions"` reply with `[transaction_id, label, llm_reason]`. (8) On any exception: default to `label="fraud", llm_reason="LLM error: ..."` (conservative).

Goals

Resolve genuinely ambiguous borderline cases that the XGBoost model assigns scores between 0.25 and 0.75. Provide a natural-language justification (`llm_reason`) for human review. Continuously enrich the fraud pattern memory for future batches. Never produce a silent failure: all error paths default to `"fraud"` (conservative assumption).

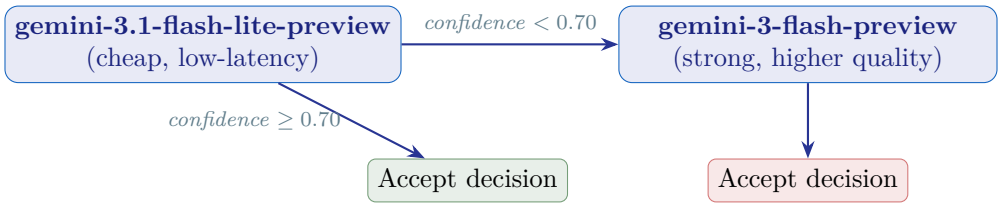
Environment

External dependencies: Google Gemini API (authenticated via `GOOGLE_API_KEY` environment variable or Application Default Credentials), Langfuse cloud (optional, authenticated via `LANGFUSE_PUBLIC_KEY` and `LANGFUSE_SECRET_KEY`). Reads/writes `artifacts/fraud_memory.json`. Singleton `genai.Client` and `Langfuse` instances are initialised lazily on first call and reused.

Self-model

CyclicBehaviour (inner class `DecideBehaviour`) with `Template(performative="request", type="llm_decide")`. The payload returned to the Orchestrator contains *only* `transaction_id`, `label`, and `llm_reason` — the Orchestrator merges these back into the full row data in `FinalizeState`.

Two-Tier Gemini Escalation



Evidence Bundle (21 fields) — llm_orchestrator.py:93

`transaction_id` · `amount` · `description` · `transaction_type` · `payment_method` · `model_score` · `card_test_score` · `behav_score` · `behav_velocity_1h` · `behav_new_recipient` · `behav_large_jump` · `geo_score` · `geo_high_risk_country` · `geo_impossible_travel` · `nlp_score` · `nlp_keyword_score` · `nlp_phishing` · `net_score` · `net_shared_device` · `net_community_risk` · `is_night`

Langfuse Tracing

Every call to `llm_decide()` opens a Langfuse **trace** named `"fraud-llm-decide"` with `metadata={n_rows: N}`. Within the trace, each transaction generates one `Generation("llm-cheap")` and optionally a second `Generation("llm-strong")` on escalation. Safety wrappers `_lf_trace`, `_lf_generation`, and `_lf_end` all check for `None` before calling Langfuse, making the entire integration optional: the system operates identically with or without Langfuse credentials. After all rows are processed, `lf.flush()` is called to

ensure batched events are sent before the function returns.

Fraud Memory Feedback Loop

Memory update — llm_orchestrator.py:188

```
new_patterns = [r["llm_reason"] for r in results if r["label"] == "fraud" and r.get("llm_reason")]
if new_patterns:
    existing = memory.get("fraud_patterns", [])
    memory["fraud_patterns"] = list(set(existing + new_patterns))[-50:]
    save_fraud_memory(memory)
# Next batch: memory["fraud_patterns"] injected into build_system_prompt(memory)
```

PROPERTIES OF THE ENVIRONMENT

Theoretical Classification

The MAS environment is analysed using the standard Russell & Norvig dimensions for intelligent agent environments:

Dimension	Classification	Opposite	Justification
Observability	Partially observable	Fully observable	Each agent receives only columns relevant to its domain; no agent sees the full database, other agents' internal state, or the Orchestrator's context dict
Determinism	Stochastic	Deterministic	Human fraud intent is unpredictable; LLM outputs are stochastic (temperature=0 reduces but does not eliminate variance); feature values depend on data quality
Episodicity	Episodic (per batch) + quasi-sequential	Purely sequential	Each batch is a self-contained episode; however, the LLM fraud memory and pre-loaded model/encoders introduce limited cross-episode state
Dynamics	Dynamic	Static	New transactions arrive continuously; fraud memory evolves after each batch; attacker behaviour adapts over time; external APIs (Gemini) may change
Continuity	Mixed	Pure discrete / continuous	Message events are discrete; feature values (amounts, distances, timestamps, probabilities) are continuous
Agent count	Multi-agent (cooperative)	Single-agent	8 agents with distinct roles share the same environment concurrently; they are cooperative (no competition for resources)
Accessibility	Accessible via interface	Inaccessible	All agents interact with the environment through well-defined SPADE message interfaces; no agent has direct database access
Predictability	Partially predictable	Unpredictable	Pipeline structure and agent interactions are deterministic; fraud patterns and LLM outputs are not

Environmental Components

Component	Format / Location	Role in MAS
Transaction stream	CSV, `data/transactions.csv`	primary input; contains amount, timestamp, sender_id, recipient_id, description, transaction_type, payment_method
User profile store	JSON, `data/users.json`	Merged on `user_id`; provides balance_after, account age, home lat/lon, device_id, phone, email
Location records	JSON, `data/locations.json`	Merged on `transaction_id`/`user_id`; provides per-transaction lat, lon, country ISO2
SMS records	JSON, `data/sms.json`	Aggregated and merged as `sms_text` per transaction/user; consumed by NLPAgent
Email records	JSON, `data/mails.json`	Subject+body aggregated as `mail_text`; consumed by NLPAgent
XGBoost model	Pickle, `models/xgb_model.pkl`	Loaded by ModelAgent at `setup()`; absent → rule-based fallback
Label encoders	Pickle, `artifacts/label_encoders.pkl`	Loaded by FeatureAgent at `setup()`; used for categorical encoding
Feature column list	Pickle, `artifacts/feature_cols.pkl`	Loaded by ModelAgent; ensures exact column order for XGBoost inference
Fraud memory	JSON, `artifacts/fraud_memory.json`	Persistent LLM-learned pattern list; read/written by LLMAgent each batch
SPADE/asyncio bus	In-process or XMPP	Shared message-passing infrastructure; `_MAILBOXES[jid]: asyncio.Queue`
Langfuse cloud	HTTPS API	External LLM call tracing; optional; credentials from environment variables
Google Gemini API	HTTPS API	LLM inference for borderline cases; requires `GOOGLE_API_KEY`

Configuration Parameters (`fraud_mas/config.py`)

Parameter	Default	Effect
`FRAUD_THRESHOLD_HIGH`	0.75	Scores $\geq$ this → auto-fraud, no LLM
`FRAUD_THRESHOLD_LOW`	0.25	Scores $\leq$ this → auto-legit, no LLM
`LLM_CONFIDENCE_MIN`	0.70	Cheap model confidence below this triggers strong model escalation
`AGENT_TIMEOUT`	30 s	Per-agent message receive timeout; $\times 2$ for LLMAgent
`XMPP_SERVER`	localhost	SPADE XMPP server hostname
`XMPP_PASSWORD`	fraud123	XMPP account password for all agents
`LLM_MODEL_CHEAP`	gemini-3.1-flash-lite	First-tier Gemini model (fast, cheap)
`LLM_MODEL_STRONG`	gemini-3-flash-preview	Second-tier Gemini model (escalation)
`LANGFUSE_HOST`	cloud.langfuse.com	Langfuse observability endpoint
`AMOUNT_LOG_SHIFT`	1.0	Shift added before log transform: $\ln(x + \text{shift})$
`TOP_N_MERCHANTS`	50	LabelEncoder cardinality cap for merchant column
`TOP_N_COUNTRIES`	30	LabelEncoder cardinality cap for country column

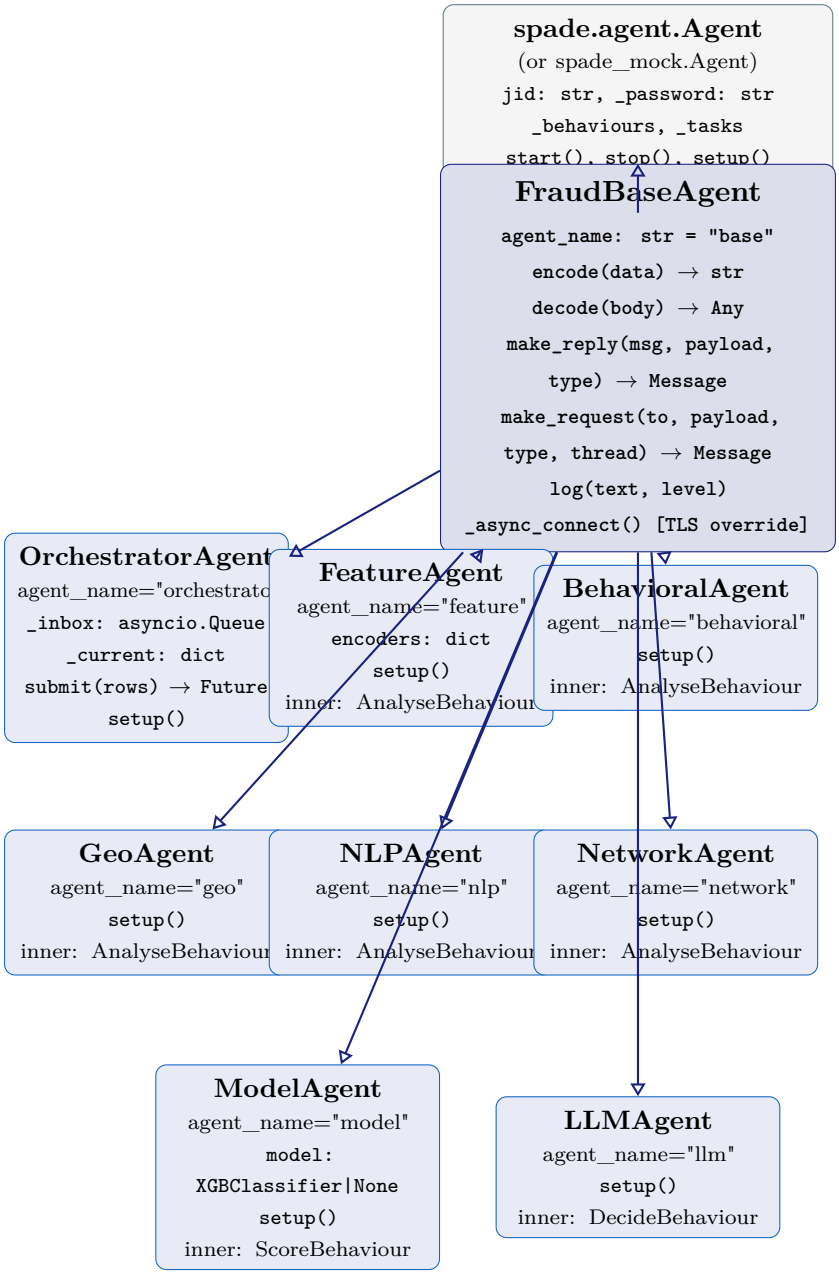
Environment Constraints and Failure Modes

- **Missing data**: All feature and signal computation functions handle absent columns with safe defaults (zero values), ensuring the pipeline never hard-fails on sparse records.

- **Agent timeout**: Any specialist agent failing to reply within `AGENT_TIMEOUT` results in zero-valued signals for that agent's columns; the ModelAgent then scores on reduced signal completeness.
- **LLM unavailability**: If `GOOGLE_API_KEY` is absent, `llm_decide()` returns all review rows as `"fraud"` with reason `"LLM skipped"`. If the API call fails, the same conservative default applies.
- **Model absence**: If `xgb_model.pkl` does not exist at `setup()`, `self.model=None` and the rule-based weighted ensemble is used as fallback.
- **Langfuse unavailability**: All Langfuse calls are wrapped in try/except; failures produce `None` handles which all downstream wrappers ignore silently.
- **XMPP unavailability**: The `_spade_compat.py` module falls back to the asyncio mock automatically on `ImportError`, so the system runs identically on Windows/Python 3.13.

APPLICATION DESIGN — CLASS DIAGRAM

Core Inheritance Hierarchy



SPADE Infrastructure Classes

Class	Key Attributes	Key Methods
`Message`	`to, sender, body, thread, _metadata: dict`	`set_metadata(k,v)`, `get_metadata(k)`, `make_reply()`→`Message`
`Template`	`_metadata: dict, thread: str|None`	`set_metadata(k,v)`, `matches(msg)`→`bool`
`_BehaviourBase`	`agent, _template, _queue: asyncio.Queue, _running`	`send(msg)`, `receive(timeout)`→`Message|None`, `_run_loop()`
`CyclicBehaviour`	(inherits _BehaviourBase)	`run()` [abstract], `_run_loop()` [loops while alive]
`State`	`_next_state: str|None`	`run()` [abstract], `set_next_state(name)`
`FSMBehaviour`	`_states: dict[str,State], _transitions: dict[str,set], _initial: str`	`add_state(name,state,initial)`, `add_transition(src,dst)`, `_run_loop()` [FSM loop]
`Agent (mock)`	`_jid, _running, _behaviours, _tasks`	`start(auto_register)`, `stop()`, `setup()`, `add_behaviour(beh,tmpl)`, `_dispatch_loop()`

Analysis Module Functions (called by agents)

Module :: Function	Signature and Role
`features :: engineer_features`	`(df, encoders|None, fit:bool)` → `(df, encoders)` — full feature engineering
`features :: get_feature_matrix`	`(df)` → `df[FEATURE_COLS]` — slim feature-only view
`features :: _rolling_tx_count`	`(df, sender_col, hours)` → `Series` — vectorised rolling count
`behavioral :: compute_behavioral_signals`	`(df)` → `df` — adds 6 behavioral signal columns
`geo :: compute_geo_signals`	`(df)` → `df` — adds 4 geo signal columns
`nlp_risk :: compute_nlp_signals`	`(df)` → `df` — adds 8 NLP signal columns
`network_analysis :: compute_network_signals`	`(df)` → `df` — adds 5 network signal columns
`model :: train`	`(df, label_col)` → `XGBClassifier` — fits and saves model
`model :: predict_proba`	`(df, model|None)` → `np.ndarray` — returns fraud probability
`model :: rule_based_score`	`(df)` → `np.ndarray` — weighted ensemble fallback
`model :: apply_thresholds`	`(scores)` → `list[str]` — fraud/legit/review classification
`model :: evaluate`	`(df, scores, label_col)` → `dict` — ROC AUC + F1
`llm_orchestrator :: llm_decide`	`(rows, memory_path|None)` → `list[dict]` — two-tier Gemini
`llm_orchestrator :: build_system_prompt`	`(memory)` → `str` — injects fraud pattern memory
`llm_orchestrator :: build_evidence_bundle`	`(row)` → `str` — 21-field JSON evidence
`llm_orchestrator :: _call_llm`	`(model_name, system, user_msg)` → `dict` — single Gemini call
`data_io :: load_and_merge_dataset`	`(txn, users, locs, sms, mails)` → `df` — full ingestion
`data_io :: load_model`	`()` → `XGBClassifier|None` — deserialise pickle
`data_io :: load_label_encoders`	`()` → `dict` — deserialise encoders
`data_io :: load_fraud_memory`	`(path|None)` → `dict` — deserialise JSON
`data_io :: save_fraud_memory`	`(memory, path|None)` — serialise JSON

Pipeline Wrapper Classes

Module :: Function	Description
`pipeline :: run_pipeline`	`(df, model, encoders)` → `df` — sequential FeatureAgent call, parallel `ThreadPoolExecutor(4)` for Behavioral/Geo/NLP/Network, sequential ModelAgent and optional LLMAgent
`spade_pipeline :: run_spade_pipeline`	`async (df)` → `df` — creates all 8 agents, `await agent.start()`, submits batch, `await Future(timeout=300)`, `await agent.stop()`
`spade_pipeline :: run_spade_pipeline_sync`	`(df)` → `df` — runs async pipeline in a dedicated thread via `asyncio.run()`, safe for Streamlit/tornado event loops

COMMUNICATION AND INTERACTION MODEL

Message Protocol Specification

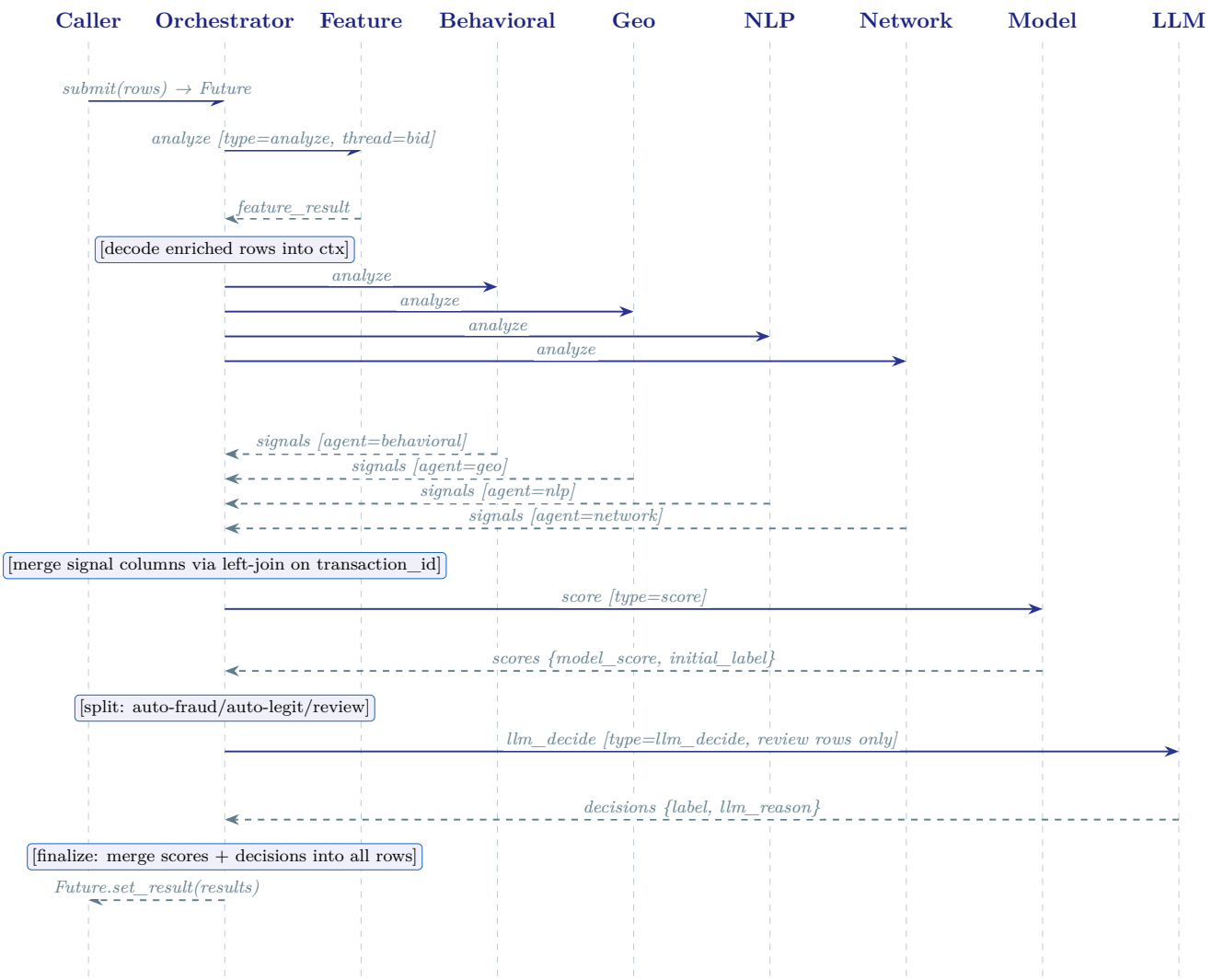
All inter-agent communication uses a uniform message envelope. Every field is mandatory; `body` carries the semantic payload.

Field	Source	Semantics
`to`	`Message.to`	Recipient JID string, e.g. `feature_agent@localhost`; maps to a mailbox queue
`sender`	`Message.sender`	Originator JID; populated by `_BehaviourBase.send()` if empty
`thread`	`Message.thread`	Batch UUID (8 hex chars); set on request, preserved by `make_reply()` via `msg.make_reply()`
`body`	`Message.body`	JSON-encoded payload; decoded with `FraudBaseAgent.decode()`
`metadata["performative"]`	metadata dict	`"request"` (Orchestrator → agent) or `"inform"` (agent → Orchestrator)
`metadata["type"]`	metadata dict	Semantic type (see full catalogue below)
`metadata["agent"]`	metadata dict	Human-readable sender name; used by Orchestrator to identify parallel reply source

Full Message Type Catalogue

type	Direction	performative	body payload
`analyze`	Orchestrator → FeatureAgent, Behavioral, Geo, NLP, Network	request	`list[dict]` of transaction rows
`feature_result`	FeatureAgent → Orchestrator	inform	`list[dict]` with 20+ feature columns appended
`signals`	Behavioral/Geo/NLP/Network → Orchestrator	inform	`list[dict]` containing only `transaction_id` + agent-specific signal columns
`score`	Orchestrator → ModelAgent	request	`list[dict]` of fully enriched rows (features + signals)
`scores`	ModelAgent → Orchestrator	inform	`list[{transaction_id, model_score: float, initial_label: str}]`
`llm_decide`	Orchestrator → LLMAgent	request	`list[dict]` of review-only rows with `model_score` attached
`decisions`	LLMAgent → Orchestrator	inform	`list[{transaction_id, label: str, llm_reason: str}]`

Interaction Sequence Diagram



Timeout and Fault-Tolerance Protocol

Phase	Timeout	Fault handling
FEATURE state	30 s (`AGENT_TIMEOUT`)	Timeout: log warning, pass raw rows through unchanged to PARALLEL
PARALLEL state	30 s (single deadline window for all 4)	Timeout: any missing agents logged as `"warning"`; missing signal columns filled with 0 via left-join NaN → 0
SCORE state	30 s	Timeout: `ctx["scores"] = []`; all rows get `model_score=0.5, initial_label="review"` → all sent to LLM
LLM_REVIEW state	60 s (2× AGENT_TIMEOUT)	Timeout: synthesise `{label:"fraud", llm_reason:"timeout"}` for every review row
Agent internal	—	Each agent wraps its computation in try/except; exception → default payload logged at ERROR level
LLM API call	—	`json.JSONDecodeError` or any Exception → `{decision:"fraud", confidence:0.5, reason:str(exc)}`

Message Correlation and Routing

Thread-based correlation: The Orchestrator generates a UUID per batch (`batch_id = str(uuid.uuid4())[:8]`) and attaches it as `message.thread` on every outgoing request. All agents preserve the thread in their replies via `msg.make_reply()` which copies `self.thread`. In `ParallelState`, the Orchestrator collects replies by checking `reply.metadata.get("type") == "signals"` and `reply.metadata.get("agent") in _PARALLEL_AGENTS`; it does not filter on thread explicitly in the parallel collect loop, relying instead on the queue isolation. In `FeatureState` and `ScoreState`, a `Template(type="feature_result",`

`thread=bid)` filter is applied.

Dispatch routing (asyncio mock): The mock `_dispatch_loop` reads messages from `_MAILBOXES[self._jid]` and routes each to the first behaviour whose `Template.matches(msg)` returns True. Unmatched messages are logged at DEBUG and discarded. Each behaviour owns a private `asyncio.Queue _queue`; FSM states share the FSM's single queue.

Dispatch routing (real SPADE / XMPP): SPADE's internal transport layer delivers XMPP stanzas to the agent's inbox queue. SPADE's behaviour scheduler routes messages to behaviours based on registered templates using the same match/route pattern.

SPADE Dual-Backend Comparison

Aspect	Real SPADE (XMPP)	asyncio Mock (in-process)
Transport	XMPP/slixmpp over TCP port 5222	`asyncio.Queue` per JID
Addressing	Jabber IDs (JIDs); registered on Prosody server	Same JID strings resolve to `_MAILBOXES[jid]`
Distribution	Agents can run on separate machines/processes	In-process only; all agents share one event loop
Auth/TLS	PLAIN auth; TLS disabled via `_async_connect()` override	No auth required
Platform	Linux/macOS, Python $\leq$ 3.12	Windows, Python 3.13, any platform
Message type	XMPP stanzas serialised as XML	Python `Message` objects passed by reference
Code changes	None (same agent classes)	None (drop-in replacement via `_spade_compat.py`)
Concurrency	Separate asyncio loops or threads per agent	All agents co-scheduled on a single asyncio event loop

Complete Data Flow Summary

Step	Actor	Input	Output Columns Added	Next step
1	`load_and_merge_dataset`	5 raw files (CSV + JSON)	all merged columns	submit to Orchestrator
2	FeatureAgent	raw tx list	20+ feature columns	PARALLEL fan-out
3a	BehavioralAgent	enriched rows	`behav_*` (6 cols)	→ Orchestrator merge
3b	GeoAgent	enriched rows	`geo_*` (4 cols)	→ Orchestrator merge
3c	NLPAgent	enriched rows	`nlp_*` (8 cols)	→ Orchestrator merge
3d	NetworkAgent	enriched rows	`net_*` (5 cols)	→ Orchestrator merge
4	ModelAgent	fully enriched rows	`model_score, initial_label`	split by threshold
5	LLMAgent	review rows only	`label, llm_reason`	→ Orchestrator finalize
6	OrchestratorAgent	all results	`label, llm_reason` (final)	Future resolved

SPADE FRAMEWORK INTEGRATION

Dual Backend Architecture

SPADE (Smart Python Agent Development Environment) provides the agent lifecycle, behaviour types, and XMPP-based transport. Because slixmpp wheels are unavailable on Windows and Python 3.13, the project implements a full drop-in replacement in `fraud_mas/agents/spade_mock.py`. The compatibility shim in `fraud_mas/agents/_spade_compat.py` selects the backend at import time:

_spade_compat.py

```python
try:
    from spade.agent import Agent
    from spade.behaviour import CyclicBehaviour, FSMBehaviour, State
    from spade.message import Message
    from spade.template import Template
    SPADE_REAL = True
except ImportError:
    from fraud_mas.agents.spade_mock import Agent, CyclicBehaviour, FSMBehaviour, State, Message, Template
    SPADE_REAL = False
```

All eight agent modules import exclusively from `_spade_compat`, so they are **backend-agnostic**. Zero code changes are required when switching between real XMPP and the asyncio mock.

Mock Backend: Global Mailbox Bus

spade_mock.py — global bus and dispatch (lines 24–249)

```python
_MAILBOXES: dict[str, asyncio.Queue] = {}   # JID  raw inbox

def _get_mailbox(jid: str) -> asyncio.Queue:
    if jid not in _MAILBOXES: _MAILBOXES[jid] = asyncio.Queue()
    return _MAILBOXES[jid]

# Agent._dispatch_loop()  runs as asyncio.Task per agent
async def _dispatch_loop(self) -> None:
    inbox = _get_mailbox(self._jid)
    while self._running:
        try:
            msg = await asyncio.wait_for(inbox.get(), timeout=0.1)
        except asyncio.TimeoutError:
            continue
        for beh, tmpl in self._behaviours:
            if tmpl is None or tmpl.matches(msg):
                await beh._queue.put(msg); break

# Template.matches()  used for routing
def matches(self, msg: Message) -> bool:
    if self.thread is not None and msg.thread != self.thread: return False
    return all(msg.get_metadata(k) == v for k, v in self._metadata.items())
```

Agent Lifecycle

spade_pipeline.py — full agent lifecycle

```python
async def run_spade_pipeline(df: pd.DataFrame) -> pd.DataFrame:
    agents = [orchestrator, feature, behavioral, geo, nlp, network, model, llm]
    for a in agents:
        await a.start(auto_register=True)  #  setup() + _dispatch_loop + behaviour tasks

    future = orchestrator.submit(df.to_dict(orient="records"))
    result_rows = await asyncio.wait_for(future, timeout=300)

    for a in agents:
        await a.stop()  # cancels all asyncio.Tasks, drops mailbox
    return pd.DataFrame(result_rows)

def run_spade_pipeline_sync(df: pd.DataFrame) -> pd.DataFrame:
    return asyncio.run(run_spade_pipeline(df))  # isolated event loop (thread-safe for Streamlit)
```

XMPP Server Configuration

For real SPADE deployment, the project ships a `prosody.cfg.lua` and a `docker-compose.yml` that start a local Prosody XMPP server. All agents authenticate with password `fraud123` and disable TLS via the `FraudBaseAgent._async_connect()` override:

```
async def _async_connect(self):
    self.client.enable_starttls = False; self.client.enable_direct_tls = False
    self.client.enable_plaintext = True; self.client["feature_mechanisms"].unencrypted_plain = True
    await super()._async_connect()
```

Agent JIDs and XMPP Addresses

Agent	JID (XMPP_SERVER=localhost)	Notes
OrchestratorAgent	`orchestrator@localhost`	FSM; single instance per pipeline run
FeatureAgent	`feature_agent@localhost`	Stateful (encoders); single instance
BehavioralAgent	`behavioral_agent@localhost`	Stateless; could run multiple instances
GeoAgent	`geo_agent@localhost`	Stateless; could run multiple instances
NLPAgent	`nlp_agent@localhost`	Stateless; could run multiple instances
NetworkAgent	`network_agent@localhost`	Stateless; could run multiple instances
ModelAgent	`model_agent@localhost`	Stateful (XGBoost model); single instance
LLMAgent	`llm_agent@localhost`	Stateful (Gemini client, Langfuse); single instance

LANGFUSE OBSERVABILITY INTEGRATION

Purpose and Architecture

Langfuse is an open-source LLM engineering platform providing tracing, evaluation, and prompt management. In this MAS, Langfuse captures every call made by the LLMAgent to the Google Gemini API. This enables post-hoc analysis of: model selection (cheap vs. strong escalation rates), decision distributions (fraud/legit ratios), confidence score distributions, latency per model tier, token consumption, and the growth of the fraud memory over time.

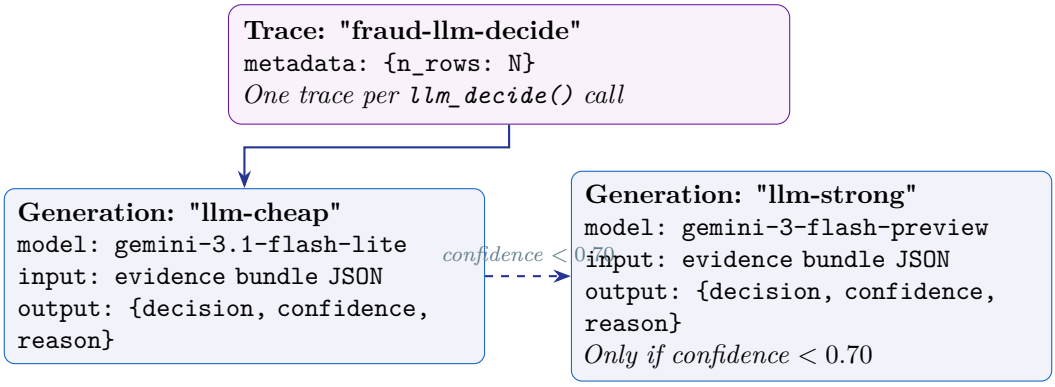
Initialisation and Credentials

Lazy singleton initialisation — llm_orchestrator.py:33

```python
_langfuse: Langfuse | None = None
_genai_client: genai.Client | None = None

def _get_langfuse() -> Langfuse | None:
    global _langfuse
    if _langfuse is None and LANGFUSE_PUBLIC_KEY and LANGFUSE_SECRET_KEY:
        try:
            _langfuse = Langfuse(
                public_key=LANGFUSE_PUBLIC_KEY,  # env: LANGFUSE_PUBLIC_KEY
                secret_key=LANGFUSE_SECRET_KEY,  # env: LANGFUSE_SECRET_KEY
                host=LANGFUSE_HOST,              # env: LANGFUSE_HOST (default: cloud.langfuse.com)
            )
        except Exception:
            pass  # fail silently  tracing disabled
    return _langfuse
```

If either key is absent or the `Langfuse()` constructor raises, `_langfuse` remains `None` and all trace/generation calls are no-ops via the safety wrappers.

Trace and Generation Structure



Safety Wrappers

Null-safe Langfuse wrappers — llm_orchestrator.py:47

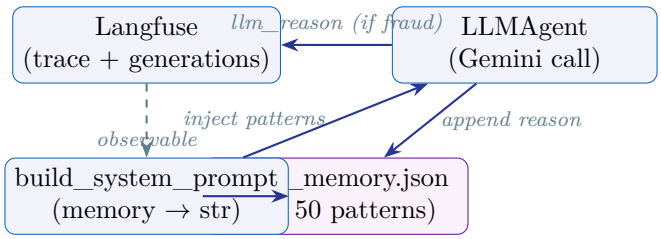
```python
def _lf_trace(lf, **kwargs):
    if lf is None: return None
    try: return lf.trace(**kwargs)
    except Exception: return None

def _lf_generation(trace, **kwargs):
    if trace is None: return None
    try: return trace.generation(**kwargs)
    except Exception: return None

def _lf_end(gen, output):
    if gen is None: return
    try: gen.end(output=output)
    except Exception: pass
```

What Langfuse Captures

Observable	Captured field	Analytical use
Batch size	`trace.metadata.n_rows`	Monitor batch distribution over time
Model tier	`generation.name` ("llm-cheap" / "llm-strong")	Measure escalation rate
Full input prompt	`generation.input` (evidence bundle JSON)	Audit which signals drove decisions
Full LLM output	`generation.output` (decision JSON)	Validate decision consistency
Latency	Auto: `generation()` → `generation.end()`	Compare cheap vs. strong latency
Token usage	Auto: captured from Gemini API response object	Cost and budget monitoring
Confidence dist.	Derived from output JSON in Langfuse dashboards	Identify poorly-calibrated decisions
Escalation events	Presence of "llm-strong" generation per trace	Model quality regression alerts

Fraud Memory Feedback Loop and Langfuse Integration

The fraud memory creates a continuous learning loop: LLM fraud decision reasons are stored in `fraud_memory.json` and injected into the system prompt for subsequent batches. All interactions in this loop are observable in Langfuse:

1. Batch k: Langfuse captures `input` (which includes `known_patterns` from memory) and `output.reason`
2. Post-batch: new fraud reasons from `output.reason` are appended to `fraud_patterns`; list capped at 50 via `list(set(existing + new))[-50:]`
3. Batch $k+1$: updated `fraud_patterns` injected into `build_system_prompt(memory)`, visible in Langfuse trace `input`

This means the full provenance of every LLM decision, including which prior fraud patterns influenced it, is traceable in Langfuse without any additional instrumentation.